

**APPLICATION- SPECIFIC ACCESS
POLICIES USING ZSCALER PRIVATE
ACCESS (ZPA).**

Project Title:

Application specific access policies using Zscaler private access (ZPA)

Program: NAAN MUDHALVAN

Course: Cyber security

Team Members:

Member 1: MANIKANDAN S

Member 2: KISHORE M

Member 3: MOHAMMAD FASITH M

Member 4: SANJAY S

Member 5: YESWANTH S.P

INSTITUTION : KATHIR COLLEGE OF ARTS AND SCIENCE

GUIDE: Ms.S.Nandhini

DATE: 08/03/2026

INTRODUCTION:

In today's rapidly evolving digital environment, organizations are increasingly exposed to sophisticated cyber threats and security breaches. As businesses adopt cloud computing, remote work environments, and mobile technologies, traditional methods of protecting corporate networks are no longer sufficient. Cyber attackers constantly exploit vulnerabilities in outdated security architectures, making it essential for organizations to implement modern and intelligent security solutions that can effectively detect, prevent, and respond to threats.

Historically, many organizations relied on a perimeter-based security model, often referred to as the "castle-and-moat" approach. In this model, users inside the corporate network were automatically trusted, while external users were considered untrusted. However, with the widespread adoption of cloud applications, remote working models, and Bring Your Own Device (BYOD) policies, the traditional network perimeter has effectively disappeared. Employees now access corporate applications from multiple locations, networks, and devices, making the conventional security model inadequate for protecting sensitive organizational resources.

To address these challenges, modern cybersecurity strategies have shifted towards the Zero Trust security model, which operates on the principle of "never trust, always verify." Instead of assuming that users inside a network are trustworthy, Zero Trust continuously validates every access request based on multiple contextual factors such as user identity, device health, location, and the sensitivity of the requested application. This approach significantly reduces the risk of unauthorized access, data breaches, and insider threats.

This project focuses on enforcing least-privilege access through identity-aware and application-specific access policies using Zscaler Private Access (ZPA). ZPA is a cloud-delivered Zero Trust Network Access (ZTNA) solution that allows organizations to provide secure access to private applications without exposing them to the public internet. Unlike traditional VPN-based access, ZPA connects users directly to specific applications rather than the entire network, ensuring that users can only access the resources necessary for their roles.

The proposed system evaluates multiple contextual parameters before granting access, including user identity, role-based permissions, device security posture, geographical location, and the risk level associated with the requested application. Based on these factors, the system dynamically determines the appropriate level

of access. For instance, the system may grant full access, limit certain functionalities, require additional authentication such as multi-factor authentication (MFA), or completely block the request if suspicious behavior is detected.

By implementing context-aware and identity-driven access policies, this project aims to enforce the principle of least privilege, ensuring that users receive only the minimum level of access required to perform their tasks. This approach minimizes the attack surface, prevents lateral movement within networks, and significantly enhances organizational security.

Ultimately, the goal of this project is to demonstrate how modern Zero Trust technologies such as Zscaler Private Access (ZPA) can provide a secure, scalable, and frictionless access experience for users while protecting critical organizational resources. By intelligently evaluating access requests in real time, organizations can maintain a strong security posture while enabling productivity for a modern, distributed workforce

ABSTRACT:

The rapid growth of cloud computing, mobile devices, and hybrid work environments has transformed how organizations access and manage their digital resources. However, these advancements have also weakened the traditional network perimeter, making conventional security models less effective. Employees and users now access organizational applications from different locations, devices, and networks, which increases the risk of cyber threats such as unauthorized access, compromised endpoints, and insider attacks. As a result, traditional access control methods that rely on static policies or simple allow/deny decisions are no longer sufficient to protect sensitive organizational data.

To address these challenges, organizations are adopting modern security frameworks based on the **Zero Trust model**, which follows the principle of *never trust, always verify*. This project focuses on implementing a **context-aware and identity-based access control system using Zscaler Private Access (ZPA)**. The system evaluates multiple contextual factors such as **user identity, device security posture, geographical location, and the risk level of the requested application** before granting access.

The proposed solution enforces **least-privilege access policies**, ensuring that users are allowed to access only the applications required for their specific roles. By defining granular and application-specific access rules, the system dynamically determines whether an access request should be granted, restricted, or denied. For example, a trusted employee accessing a low-risk application from a secure and managed device may be granted seamless access, while attempts from unknown devices or risky locations may trigger additional verification such as **multi-factor authentication (MFA)** or access restrictions.

The expected outcome of this project is the development of a secure and adaptive access control framework that minimizes the organizational attack surface. By evaluating access requests in real time and applying intelligent security policies, the system enhances protection while maintaining a smooth user experience. This approach demonstrates how **Zero Trust solutions like Zscaler Private Access** can enable secure and efficient access to enterprise applications in modern distributed environments.

Use Case Scenarios

Scenario 1: Finance Department Access to ERP System

A senior accountant in the finance department needs to access the corporate Enterprise Resource Planning (ERP) system to process monthly payroll. The user connects from a remote location using their company-managed laptop. Upon launching the ZPA Client Connector, they authenticate via Azure Active Directory with multi-factor authentication (MFA). ZPA evaluates the user's identity, group membership (Finance_ERP_Users), device posture (antivirus active, disk encryption enabled), and connection context. The policy engine determines that the user is authorized and brokers a secure, direct micro-tunnel to the ERP application only. The user can interact with the ERP interface, but has no visibility or network-level access to any other internal systems, such as development servers or the HR database.

Output:

1. User authenticated and authorized based on identity and group membership
2. Direct micro-tunnel established specifically to the ERP application
3. All other internal applications and network segments remain invisible and inaccessible

Scenario 2: Contractor Access to Specific Project Repository

An external software consultant is hired for a three-month project and requires access only to a specific Git repository hosted on an internal server. The contractor is onboarded as a guest user in the identity provider and assigned to the "Project_X_Contractors" group. When the contractor connects using the ZPA client, the system verifies their identity and checks the device for basic security compliance (OS patch level, firewall enabled). ZPA enforces a policy that grants access exclusively to the Git repository application (port 22/SSH or port 443/HTTPS) during business hours (9 AM to 5 PM). The contractor can clone, push, and pull code, but cannot discover or connect to any other internal resources, including the corporate intranet or file servers.

Output:

4. Time-bound and role-specific access enforced
5. Contractor restricted to a single application (Git repository)
6. Lateral movement to adjacent systems completely blocked

Scenario 3: Executive Access to Confidential Board Documents

A company executive needs to access a confidential folder containing board meeting materials hosted on an internal file server. The executive connects from a tablet device while traveling internationally. ZPA integrates with the mobile device management (MDM) solution to verify that the tablet is compliant (not jailbroken, passcode enabled, encryption active). Upon successful authentication, ZPA evaluates the policy: "If user is in 'Executive_Board' group AND device is MDM-compliant AND connection is from a trusted country (allowlist), THEN grant access to 'Board_Documents' application." The executive is connected directly to the specific file share, and all data in transit is encrypted via TLS. Any attempt to access other shares or network resources is automatically blocked and logged.

Output:

7. Context-aware access based on user role, device posture, and geographic location
8. Granular access to a specific file share (Board_Documents)
9. Comprehensive logging of access events for audit and compliance Scenario

4: IT Administrator Access to Server Management Interfaces

A systems administrator requires access to management interfaces (SSH for Linux servers and RDP for Windows servers) to perform routine maintenance. Instead of granting VPN access to the entire management network, ZPA is configured to treat each server or management interface as a separate application segment. The administrator authenticates and is presented only with the specific servers they are authorized to manage based on their IT subgroup (e.g., "Linux_Admins" see only Linux servers; "Windows_Admins" see only Windows servers). When the admin connects via SSH to a Linux server, ZPA creates a direct micro-tunnel to that specific IP and port combination. The admin cannot scan the subnet, access other servers, or initiate connections to non-authorized ports.

Output:

10. Micro-segmentation at the individual server/interface level
11. Role-based access controlling visibility of specific management consoles
12. Elimination of network-level reconnaissance capabilities for admins

Scenario 5: Mergers and Acquisitions Integration

A company acquires a smaller firm and needs to provide the acquired company's HR team with temporary access to the parent company's benefits enrollment application. The acquired company's users are added as external collaborators in the identity provider with limited attributes. ZPA policies are created that specifically grant the "Acquired_HR" group access only to the "Benefits_App" (a web application hosted internally). The acquired users install the ZPA client, authenticate, and are connected directly to the benefits application. Because ZPA provides access without network connectivity, the parent company's entire internal network remains completely hidden and protected. The acquired users cannot ping, scan, or access any other corporate resource. After six months, when the integration is complete and users are migrated to the parent identity system, the temporary policies are revoked.

Output:

13. Secure, temporary access provided without network exposure
14. Complete network invisibility protecting the parent company's infrastructure
15. Simplified deprovisioning through policy revocation rather than complex network changes

Stage – 1

Title :

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Overview

As organizations adopt cloud technologies, remote work, and mobile devices, the traditional network boundary has become less relevant. In the past, security models relied heavily on VPN-based access where users connected to the internal network and were implicitly trusted once authenticated. This model often resulted in users receiving broad network access, which increased the chances of unauthorized access, internal threats, and lateral movement across systems if credentials were compromised. Modern security strategies must therefore move beyond location-based trust and focus on verifying every access request based on identity and context.

This project addresses these challenges by implementing a **Zero Trust architecture** that follows the principle of “*never trust, always verify*.” Instead of granting access simply because a user is connected to the corporate network, the system evaluates multiple factors before allowing access to applications. These factors include the user’s identity, role, device posture, and other contextual information that help determine whether the access request should be permitted.

The solution utilizes **Zscaler Private Access (ZPA)**, a cloud-delivered platform that provides secure, identity-aware access to private applications. Unlike traditional VPN solutions, ZPA does not place users directly on the corporate network. Instead, it establishes a secure connection only to the specific application the user is authorized to access. Internal applications remain hidden from the public internet, which helps prevent common attack techniques such as network scanning or unauthorized discovery of internal resources.

Through this approach, the project enforces a **least-privilege access model**, where users and devices receive only the minimum level of access required to perform their tasks. Because access is limited to individual applications rather than the entire network, users cannot move laterally between systems. This significantly reduces the potential attack surface while improving overall security.

List of Vulnerability:

S. No.	Vulnerability Name	CWE Number
1.	Relative Path Traversal	CWE-23
2.	Active Debug Code	CWE-489
3.	Software Supply Chain Compromise	CWE-829
4.	Weak Cryptographic Implementation	CWE-327
5.	SQL Injection in Hibernate Queries	CWE-564
6.	Missing Encryption of Sensitive Data	CWE-311
7.	Use of Hard-coded Password	CWE-259
8.	Inclusion of Functionality from Untrusted Control Sphere	CWE-829
9.	Improper Output Neutralization for Logs (Log Injection)	CWE-117
10.	NULL Pointer Dereference	CWE-476

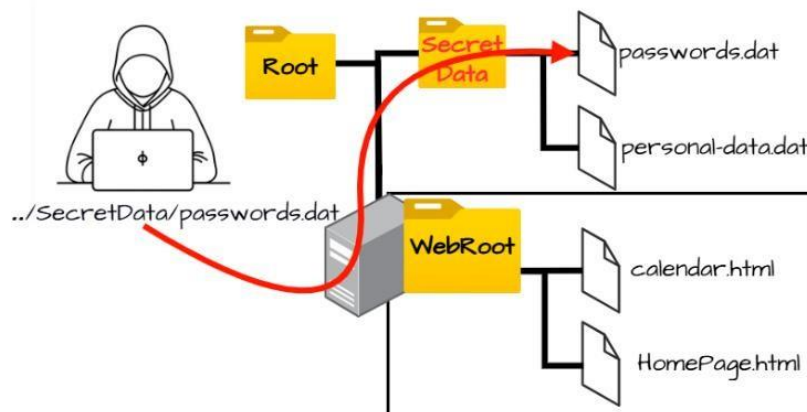
Report:

1. Vulnerability name: **Relative Path Traversal**

CWE: 23

OWASP category: A01:2021 – Broken Access Control

Description: Relative Path Traversal occurs when an application constructs file paths using user input without proper validation, allowing attackers to manipulate directory sequences like “../”, which enables them to move outside the intended directory and access sensitive files on the server.



Business Impact Attackers can gain unauthorized access to confidential system files, configuration data, or credentials, which may lead to data breaches, system compromise, operational disruption, and significant financial and reputational damage to the organization.

Recommendation:

1. **Input Validation:** Validate and sanitize all user inputs used in file paths and allow only predefined safe values.
2. **Path Canonicalization:** Normalize and verify file paths to remove traversal sequences such as “../” before accessing files.
3. **Access Control Restrictions:** Restrict file access to specific directories and enforce strict file permissions to prevent unauthorized file access.
4. such as “../” before accessing files.
5. **Access Control Restrictions:** Restrict file access to specific directories and enforce strict file permissions to prevent unauthorized file access.

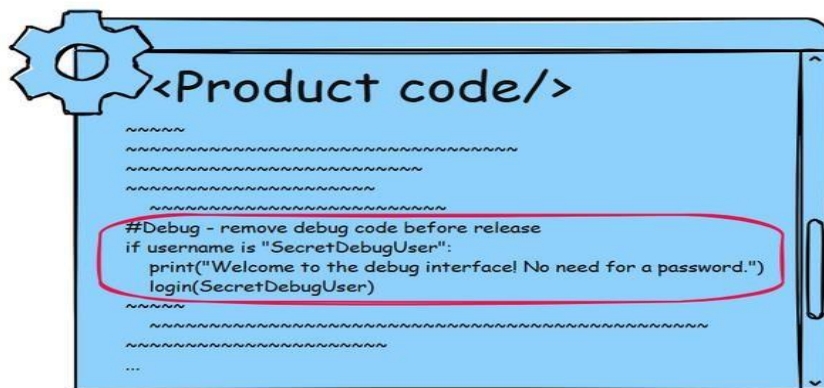
2. Vulnerability Name: [Active Debug Code](#)

CWE-489

OWASP Category: A02:2021 – Security Misconfiguration

Description: Active Debug Code occurs when a software product is released with debugging or testing code still enabled in the production environment, which can expose internal application behavior and create unintended entry points for attackers, potentially allowing them to manipulate application functions or bypass security controls.

Business Impact: If debug functionality remains active in a deployed system, attackers may exploit it to access sensitive application data, bypass authentication mechanisms, or gain elevated privileges, which could ultimately lead to full compromise of the application or underlying server infrastructure.



Recommendation:

1. **Remove Debug Code Before Deployment:** Ensure that all debugging functions, testing endpoints, and development backdoors are completely removed or disabled before deploying the application to production environments.
2. **Implement Secure Configuration Management:** Adopt strict configuration management practices to ensure that debug modes, logging levels, and development settings are not enabled in production systems.

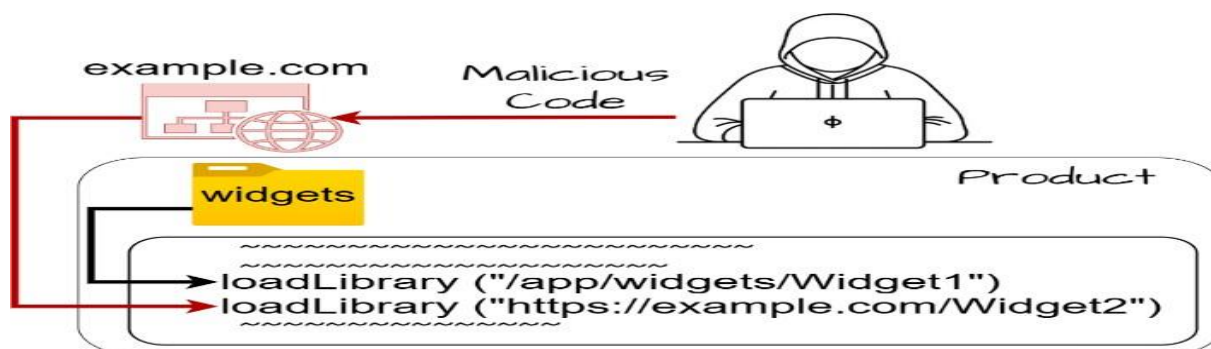
Perform Security Testing and Code Review: Conduct static code analysis, penetration testing, and security reviews to detect leftover debug code and other insecure development artifacts before releasing the application.

3. Vulnerability Name: **Software Supply Chain Compromise**

CWE-829

OWASP Category: A03:2025 – Software Supply Chain Failures

Description: Software supply chain failures occur when an application relies on third-party libraries, dependencies, or components that contain vulnerabilities or malicious code, which can compromise the security of the entire system.



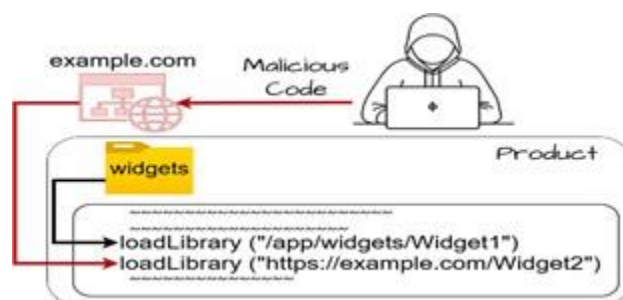
Recommendation:

1. **Dependency Verification:** Use trusted repositories and verify the integrity of third-party libraries before including them in the application.
2. **Regular Dependency Updates:** Continuously update and patch third-party components to reduce exposure to known vulnerabilities.
3. **Supply Chain Monitoring:** Use automated tools to scan and monitor software dependencies for security risks.
4. **Vulnerability Name:** **Weak Cryptographic Implementation**

CWE - 327

OWASP Category: A04:2025 – Cryptographic Failures

Description: Cryptographic failures occur when sensitive data is protected using weak encryption algorithms, improper key management, or insecure transmission mechanisms.



Business Impact: Attackers may inject malicious code through compromised dependencies, leading to system compromise, data theft, and large-scale attacks affecting multiple financial information, or personal records.

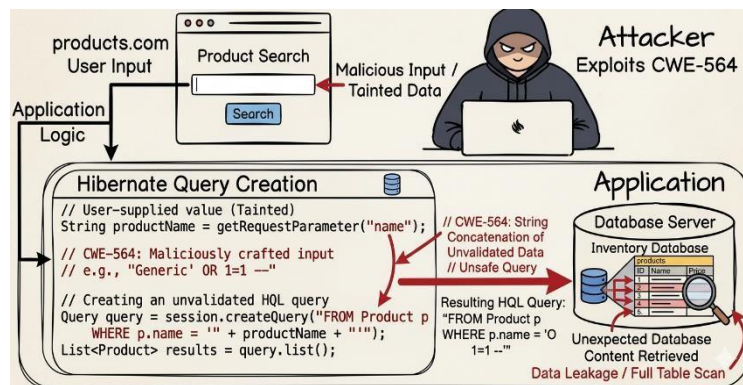
Recommendation:

1. **Use Strong Encryption:** Implement modern encryption standards such as AES-256 and TLS 1.3 to secure sensitive data.
2. **Secure Key Management:** Store cryptographic keys securely and rotate them periodically to prevent misuse.
3. **Encrypt Sensitive Data:** Ensure that all sensitive data is encrypted both in transit and at rest.

5. Vulnerability Name: **SQL Injection in Hibernate Queries**

CWE-564

OWASP Category: A05:2025 – Injection



Description: SQL Injection in Hibernate occurs when an application builds Hibernate Query Language (HQL) statements using unvalidated user input, allowing attackers to manipulate the query structure and execute unauthorized database commands through the Hibernate framework.

Business Impact: Attackers may retrieve, modify, or delete sensitive database records, potentially leading to data breaches, unauthorized access to user information, and compromise of the entire application database.

Recommendation:

Use Parameterized Queries: Always use parameter binding in Hibernate queries to prevent user input from being interpreted as executable query code.

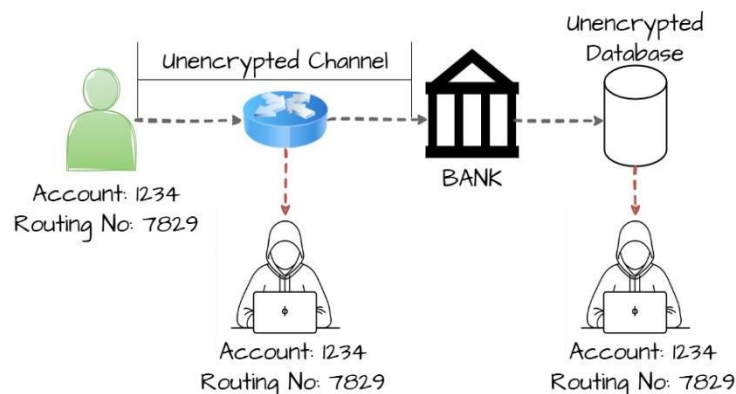
Input Validation: Validate and sanitize all user inputs before using them in HQL queries to ensure that malicious characters or commands are rejected.

Least Privilege Database Access: Configure database accounts with minimal privileges so that even if an injection occurs, the attacker cannot perform critical operations.

6. Vulnerability Name: **Missing Encryption of Sensitive**

Data CWE – 311

OWASP Category: A04:2025 – Cryptographic Failures



Description: This vulnerability occurs when sensitive information such as passwords, personal data, or financial details is stored or transmitted without proper encryption, making it accessible to unauthorized parties who may intercept or access the data.

Business Impact: Attackers may capture or read sensitive data through network interception or unauthorized system access, which can lead to data breaches, identity theft, financial loss, and reputational damage for the organization.

Recommendation:

7. **Encrypt Sensitive Data:** Use strong encryption algorithms such as AES or TLS to protect sensitive data during storage and transmission.
8. **Secure Communication Channels:** Ensure that all data exchanges occur over secure protocols like HTTPS instead of insecure protocols such as HTTP.
9. **Proper Key Management:** Store and manage encryption keys securely and rotate them periodically to prevent unauthorized access.

10.Vulnerability Name: **Use of Hard-coded Password**

CWE- 259

OWASP Category: A07- 2025 – Authentication Failures

Description: This vulnerability occurs when a password is directly embedded in the application source code or configuration files, allowing anyone who gains access to the code to easily discover and use the credentials to access the system.

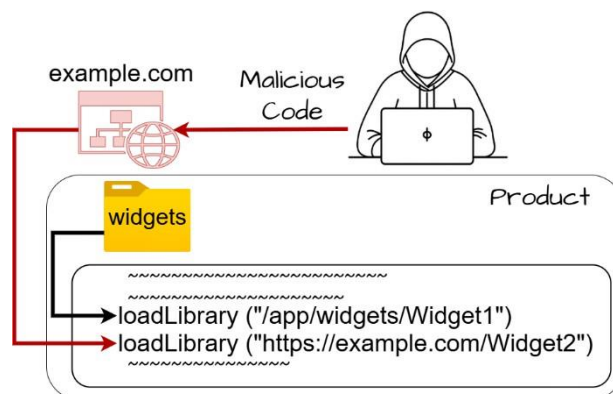


Business Impact: Attackers who obtain the hard-coded password may gain unauthorized access to applications, database Administrative functions, potentially leading to breaches, system compromise, and loss of sensitive information.

Recommendation:

- 1) **Use Secure Credential Storage:** Store passwords securely using environment variables or dedicated secret management systems instead of embedding them in source code.
- 2) **Implement Strong Authentication:** Use secure authentication mechanisms such as token-based authentication or multi-factor authentication to reduce reliance on static passwords.
- 3) **Regular Credential Rotation:** Frequently change passwords and remove hard-coded credentials from the application to minimize the risk of unauthorized access

Description: This vulnerability occurs when user-controlled input is written directly to log files without proper sanitization, allowing attackers to manipulate log entries or inject malicious content.



Business Impact: Attackers can inject malicious libraries or scripts into the application, leading to unauthorized code execution, system compromise, or large-scale supply chain attacks affecting multiple systems.

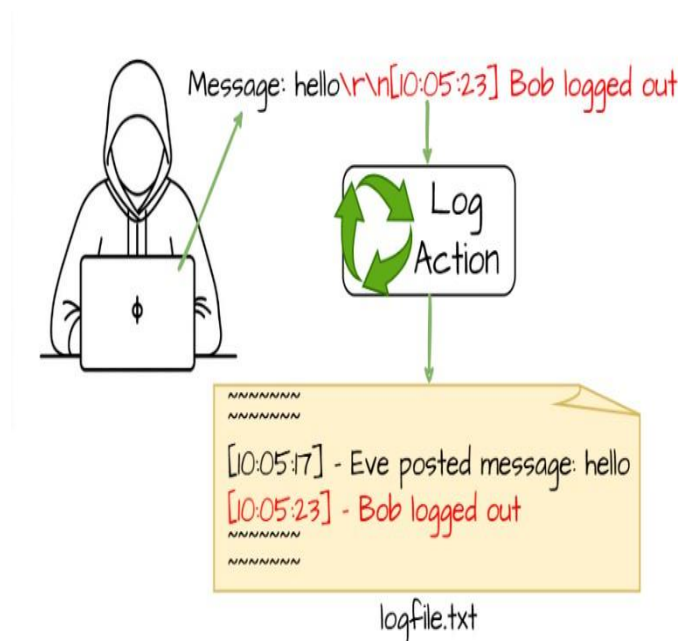
Recommendation:

- 1) **Validate External Sources:** Ensure all external libraries and modules are obtained from trusted repositories and verified before use.
- 2) **Implement Integrity Checks:** Use digital signatures or hash verification to confirm that external components have not been tampered with.
- 3) **Limit External Code Execution:** Restrict the application from dynamically loading code from unknown or untrusted sources.

11.Vulnerability Name: **Improper Output Neutralization for Logs (Log Injection)**

CWE- 117

OWASP Category: A09:2025 – Security Logging and Alerting Failure



description: This vulnerability occurs when user- controlled input is written directly to log files without proper sanitization, allowing attackers to manipulate log entries or inject malicious content.

Business Impact: Attackers can alter or forge log records, hide malicious activities, or inject misleading information, which may disrupt security monitoring and incident investigations.

Recommendation:

1. **Sanitize Log Inputs:** Filter or encode user input before writing it to log files to prevent injection attacks.
2. **Use Structured Logging:** Implement standardized logging formats that prevent manipulation of log entries.
3. **Protect Log Integrity:** Restrict access to log files and implement monitoring to detect unauthorized changes.

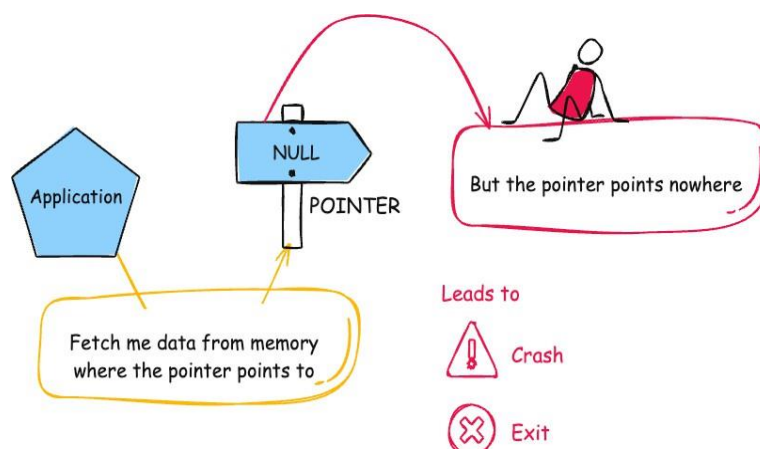
12. Vulnerability Name: NULL Pointer Dereference

CWE- 476

OWASP Category: A10:2025 – Mishandling of Exceptional Conditions

Description: This vulnerability occurs when a program attempts to access or dereference a pointer that has a NULL value, which may cause the application to crash or behave unpredictably.

Business Impact: Exploiting this vulnerability may result in system crashes, denial of service, or instability of the application, affecting availability and reliability of the system.



Recommendation:

1. **Validate Pointers Before Use:** Always check whether a pointer is NULL before accessing it in the program.
2. **Implement Proper Exception Handling:** Use error handling mechanisms to manage unexpected conditions safely.
3. **Perform Secure Code Reviews:** Conduct regular code reviews and testing to detect and fix potential NULL pointer issues.

Stage – 2

Overview: -

Nessus is a vulnerability scanner developed by Tenable, Inc. It is a remote scanner, meaning that it does not need to be installed on the target host to scan it. Nessus works by sending a variety of probes to the target host in order to identify vulnerabilities. These probes can include network traffic, operating system commands, and application requests. Nessus uses a plugin-based architecture. Plugins are small pieces of code that are designed to identify specific vulnerabilities.

There are over 1200 plugins available for Nessus, covering a wide range of vulnerabilities in operating systems, applications, and network devices. When Nessus scans a host, it first uses a discovery plugin to identify the operating system and services running on the host. Once the host has been discovered, Nessus runs a series of plugins to test for vulnerabilities.

For each vulnerability that Nessus finds, it provides a detailed report that includes the severity of the vulnerability, the steps to reproduce the vulnerability, and the steps to remediate the vulnerability. Nessus can be used to scan a variety of targets, including single hosts, networks, and web applications. It can also be used to scan specific ports and services. Nessus can be configured to run scans on a schedule or manually.

Here is a step-by-step explanation of how Nessus works:

1. Nessus connects to the target host using a variety of protocols, including TCP, UDP, and ICMP.
2. Nessus sends a series of probes to the target host in order to identify its operating system and services.
3. Nessus runs a series of plugins to test for vulnerabilities.
4. Nessus generates a report that includes the severity of each vulnerability, the steps to reproduce the vulnerability, and the steps to remediate the vulnerability.

Nessus is a valuable tool for security professionals who need to identify and remediate vulnerabilities in their networks. It is easy to use and provides comprehensive reports that can be used to prioritise remediation efforts.

Here are some of the benefits of using Nessus:

- **Comprehensive vulnerability coverage:** Nessus has over 1200 plugins available, covering a wide range of vulnerabilities in operating systems, applications, and network devices.
- **Easy to use:** Nessus has a user-friendly interface that makes it easy to configure and run scans.
- **Comprehensive reporting:** Nessus provides detailed reports that include the severity of each vulnerability, the steps to reproduce the vulnerability, and the steps to remediate the vulnerability.
- **Scalability:** Nessus can be used to scan a variety of targets, including single hosts, networks, and web applications.
- **Flexibility:** Nessus can be configured to run scans on a schedule or manually.

Nessus is a powerful tool that can be used to improve the security of any network. It is a valuable asset for any security professional.

Target website :- <http://testfire.net/>

Target ip address :- 65.61.137.117

List of vulnerability:-

s.no	Vulnerability name	Severity	plugins
1	TLS Version 1.0 Protocol Detection	MEDIUM	104743
2	TLS Version 1.1 Protocol Deprecated	MEDIUM	157288
3	SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)	LOW	83875
4	ICMP Timestamp Request Remote Date Disclosure	INFO	10114
5	Additional DNS Hostnames	INFO	46180
6	Apache Tomcat Detection	INFO	39446
7	Common Platform Enumeration (CPE)	INFO	45590
8	Device Type	INFO	54615
9	HSTS Missing From HTTPS Server	INFO	84502
10	HTTP Server Type and Version	INFO	10107

REPORT: -

1) Vulnerability Name: - [104743 - TLS Version 1.0 Protocol](#)

Detection Severity: - MEDIUM

Plugin: -04743

Port: - 443

Description: - The remote service accepts connections encrypted using TLS 1.0. TLS 1.0 has a number of cryptographic design flaws. Modern implementations of TLS 1.0 mitigate these problems, but newer versions of TLS like 1.2 and 1.3 are designed against these flaws and should be used whenever possible.

As of March 31, 2020, Endpoints that aren't enabled for TLS 1.2 and higher will no longer function properly with major web browsers and major vendors. PCI DSS v3.2 requires that TLS 1.0 be disabled entirely by June 30, 2018, except for POS POI terminals (and the SSL/TLS termination points to which they connect) that can be verified as not being susceptible to any known exploits.

Solution: - Enable support for TLS 1.2 and 1.3, and disable support for TLS 1.0.

Business Impact: - Continued use of TLS 1.0's encryption, known for its inherent cryptographic flaws, exposes the organization to heightened security risks, increasing the likelihood of data breaches and cyberattacks. This vulnerability jeopardizes compliance with industry standards, specifically PCI DSS v3.2, which mandates the complete cessation of TLS

1.0 use. Noncompliance risks regulatory penalties and erodes customer trust. Moreover, endpoints that lack support for TLS 1.2 and higher will experience impaired functionality with major web browsers and vendors, potentially leading to lost business opportunities and damage to the organization's reputation. Addressing this vulnerability is imperative to mitigate these multifaceted risks and maintain a secure and trustworthy online presence.

2) Vulnerability Name: - [157288 - TLS Version 1.1 Protocol](#)

Deprecated Severity: - MEDIUM

Plugin: -157288

Port: - 443

Description: - The remote service accepts connections encrypted using TLS 1.1. TLS 1.1 lacks support for current and recommended cipher suites. Ciphers that support encryption before MAC computation, and authenticated encryption modes such as GCM cannot be used with TLS 1.1 As of March 31, 2020, Endpoints that are not enabled for TLS 1.2 and higher will no longer function properly with major web browsers and major vendors.

Solution: -Enable support for TLS 1.2 and/or 1.3, and disable support for TLS 1.1.

Business Impact: -The continued use of TLS 1.1 with its lack of support for modern cipher suites and advanced encryption methods jeopardizes the organization's security posture, increasing vulnerability to cyber threats and potential data breaches. Additionally, endpoints not upgraded to support TLS 1.2 and higher will encounter compatibility issues with major web browsers and vendors, impacting user experience and potentially leading to missed business opportunities. Swift remediation of this vulnerability is essential to mitigate security risks and ensure seamless interactions with web browsers and vendors, preserving the organization's reputation and operational efficiency.

3) Vulnerability Name: - [83875 - SSL/TLS Diffie-Hellman Modulus <= 1024](#)

Bits (Logjam) Severity: - LOW

Plugin: - 83875

Port: - 443

Description: -The remote host allows SSL/TLS connections with one or more Diffie-Hellman moduli less than or equal to 1024 bits. Through cryptanalysis, a third party may be able to find the shared secret in a short amount of time (depending on modulus size and attacker resources). This may allow an attacker to recover the plaintext or potentially violate the integrity of connections.

Solution: - Reconfigure the service to use a unique Diffie-Hellman moduli of 2048 bits or greater.

Business Impact: - Allowing SSL/TLS connections with weak Diffie-Hellman moduli raises security concerns. An attacker with sufficient resources and computing power could compromise the shared secret relatively quickly, potentially leading to the decryption of sensitive data or the violation of connection integrity. While the severity is low, the risk of data breaches and integrity compromise should not be underestimated, and it is essential to address this vulnerability to maintain data security and user trust.

4) Vulnerability Name: - 10114 - **ICMP Timestamp Request Remote**

Date Disclosure Severity: - INFO

Plugin: - 10114

Port: - 0

Description: -The remote host answers to an ICMP timestamp request. This allows an attacker to know the date that is set on the targeted machine, which may assist an unauthenticated, remote attacker in defeating time-based authentication protocols. Timestamps returned from machines running Windows Vista / 7 / 2008 / 2008 R2 are deliberately incorrect, but usually within 1000 seconds of the actual system time.

Solution: - Filter out the ICMP timestamp requests (13), and the outgoing ICMP timestamp replies (14).

Business Impact: -This does not pose an immediate threat to business operations or data security. This vulnerability allows external parties to request the system's date and time information remotely. While it does not present a significant risk, it could potentially be used by attackers for reconnaissance purposes. Therefore, it is advisable to address this vulnerability as part of a broader security posture to reduce the risk of unauthorized data gathering and protect against potential future threats.

5) Vulnerability Name: - 46180 - Additional DNS

Hostnames Severity: - INFO

Plugin: - 46180

Port: - 0

Description: - Hostnames different from the current hostname have been collected by miscellaneous plugins. Nessus has generated a list of hostnames that point to the remote host. Note that these are only the alternate hostnames for hosts discovered on a web server. Different web servers may be hosted on name-based virtual hosts.

Solution: - If you want to test them, re-scan using the special vhost syntax, such as : www.example.com [192.0.32.10]

Business Impact: -This does not pose an immediate threat to business operations or data security. It indicates the discovery of hostnames that are associated with the remote host, particularly in cases of web servers utilizing name-based virtual hosts. While not a security risk on its own, it could be useful for understanding the server's configuration and infrastructure. Organizations should consider this information as part of their network documentation but do not need to take immediate action to mitigate it as a security threat.

6) Vulnerability Name: - 39446 -Apache Tomcat

Detection Severity: - INFO

Plugin: - 39446

Port: - 80

Description: - Nessus was able to detect a remote Apache Tomcat web server.

Solution: - n/a

Business Impact: -Is an informational finding that identifies the presence of a remote Apache Tomcat web server. While this information itself does not indicate a security threat or impact on business operations, it can be useful for network administrators and security professionals to be aware of the technologies in use on their network. Understanding the presence of an Apache Tomcat server can be valuable for maintaining and securing web applications, but it does not require immediate action to mitigate a security risk.

7) Vulnerability Name: - 45590 - Common Platform

Enumeration (CPE) Severity: - INFO

Plugin: - 45590

Port: - 0

Description: - By using information obtained from a Nessus scan, this plugin reports CPE (Common Platform Enumeration) matches for various hardware and software products found on a host. Note that if an official CPE is not available for the product, this plugin computes the best possible CPE based on the information available from the scan

Solution: - n/a

Business Impact: - This provides insights into hardware and software products identified on a host through a Nessus scan. It reports CPE matches, which are standardized identifiers for these products. This information is valuable for asset management and inventory purposes. While it doesn't signify a security threat, it aids administrators in understanding the components within their network, contributing to effective resource management and facilitating future system maintenance.

8) Vulnerability Name: - Device

Type Severity: - INFO

Plugin: - 54615

Port: - 0

Description: - Based on the remote operating system, it is possible to determine what the remote system type is (eg: a printer, router, general-purpose computer, etc).

Solution: - n/a

Business Impact: -It allows for the identification of the remote system type, such as a printer, router, or general-purpose computer, based on the remote operating system. While this doesn't pose a direct security threat, it offers valuable insights for network administrators to better understand and classify the devices in their network. This information aids in network management, optimization, and ensures that the right security measures are in place for each device type.

9) Vulnerability Name: -84502 - HSTS Missing From

HTTPS Server Severity: - INFO

Plugin: - 84502

Port: - 443

Description: -The remote HTTPS server is not enforcing HTTP Strict Transport Security (HSTS). HSTS is an optional response header that can be configured on the server to instruct the browser to only communicate via HTTPS. The lack of HSTS allows downgrade attacks, SSLstripping man-in-the-middle attacks, and weakens cookie-hijacking protections.

Solution: - Configure the remote web server to use HSTS

Business Impact: -Indicates that the remote HTTPS server does not enforce HTTP Strict Transport Security (HSTS). HSTS is a crucial security feature that, when configured on the server, instructs web browsers to communicate exclusively via HTTPS, enhancing security. The absence of HSTS potentially exposes the server to downgrade attacks, SSL-stripping man- in-themiddle attacks, and weakens protections against cookie hijacking.

10) Vulnerability Name: - 10107 - HTTP Server Type

and Version Severity: - INFO

Plugin: - 10107

Port: - 443

Description: - This plugin attempts to determine the type and the version of the remote web server.

Solution: - n/a

Business Impact: This relates to the identification of the remote web server's type and version. While this issue doesn't pose an immediate security threat, it can provide valuable information to potential attackers about the server's software stack. Knowledge of the server type and version may aid attackers in exploiting known vulnerabilities specific to that software, making it important to restrict such information from being easily accessible to enhance the server's overall security posture.

Stage – 3

Title:

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Overview:

The primary objective of this project is to implement **Zero Trust Network Access (ZTNA)** to secure user-to-application connectivity within an organization. Traditional security models rely heavily on Virtual Private Networks (VPNs) where users gain access to the internal network once they authenticate successfully. However, this model presents significant security risks because once inside the network, a compromised account can move laterally across multiple systems.

To address these limitations, modern cybersecurity frameworks adopt the **Zero Trust principle**, which operates on the concept of “*Never Trust, Always Verify.*” In this architecture, no user or device is automatically trusted, even if they are inside the corporate network. Every access request must be verified based on multiple security parameters before granting permission.

This project focuses on securing application access using Zscaler Private Access. ZPA creates a secure connection between a user and a specific application rather than allowing access to the entire network. It ensures that internal applications remain hidden from the public internet and only authorized users can access them after verification.

The system evaluates several critical factors such as user identity, device posture, application risk level, and contextual information before granting access. By enforcing strict access policies and continuous monitoring, ZTNA significantly reduces the attack surface and prevents unauthorized access attempts. This approach improves security visibility and ensures that sensitive organizational resources remain protected against cyber threats.

Context-Aware Access Control Model

Context-aware access control is a security model where access to applications or resources is granted based on contextual factors such as user identity, device security status, location, and time of access. In systems like Zscaler Private Access, the system evaluates these conditions before allowing the user to connect to an application. This ensures that access is given only when the request meets the required security policies, improving overall protection against unauthorized access.

Types of Context-Aware Access Control

1. User Identify
2. Device Posture
3. Location and Network Context
4. Application Risk Level

Each of these factors contribute to the overall security decision.

User Identity

Access is granted only after verifying the identity of the user through authentication systems such as Microsoft Azure Active Directory. This ensures that only authorized users can access specific applications.

After successful authentication,,ZPA reterives attributes including:

- Username
- Department
- Role
- Group membership
- Access privileges

These attributes allow administrators to create **identity-based access policies**. For example, only users belonging to a particular department or role are allowed to access specific applications. This identity-centric approach ensures that access decisions are based on **who the user is**, rather than where the user is connecting from.

Device Posture Assessment

Even if a user's identity is valid, an insecure or compromised device can introduce security risks. Therefore, context-aware access control also evaluates the **security posture of the device** attempting to access the application.

Zscaler Private Access integrates with endpoint security and device management tools such as:

- Microsoft Defender
- CrowdStrike
- SentinelOne
- Microsoft Intune

These integrations allow the system to verify the device's **health and compliance status** before granting access to internal applications.

- Operating system patch status
- Antivirus installation and updates
- Disk encryption status
- Firewall configuration
- Detection of rooted or jailbroken devices

If a device fails the required security checks, the system may deny access or redirect the user to a remediation portal where security issues must be resolved before access is granted. This ensures that only secure and compliant devices can access critical organizational applications.

Location and Network Context

Another important factor in context-aware access control is the location and network environment from which the user is connecting. Access requests may originate from different locations such as corporate offices, home networks, public Wi-Fi networks, or international locations. Each of these environments carries a different level of risk.

For example:

- Access from corporate networks may be considered low risk.
- Access from public Wi-Fi networks may require additional verification.
- Access from high-risk geographic regions may be blocked entirely.

This approach helps organizations ensure that users are accessing applications from trusted locations and secure network environments, thereby reducing the chances of unauthorized access.

Application Risk Classification

Not all applications have the same level of sensitivity. Therefore, applications are categorized based on their **risk level and business importance**.

Applications may be classified as:

Low-Risk Applications

- Internal knowledge portals
- Employee directories
- General information systems

Medium-Risk Applications:

- Internal collaboration tools
- File sharing systems
- Department portals

High-Risk Applications:

- Financial systems
- Customer databases
- Source code repositories
- Research data systems

Access policies are applied according to the **risk level of each application**. High-risk applications require stricter access controls such as stronger authentication and secure, compliant devices before access is granted.

Implementation Approach

The implementation of context-aware access control using Zscaler ZPA follows several key steps.

Step 1: Application Discovery

Identify all internal applications that require secure access. **Step 2:**

User Segmentation

Group users based on roles and responsibilities such as students, faculty, administrators, and IT staff. **Step**

3: Identity Integration

Connect Zscaler with the organization's identity provider to enable authentication and MFA.

Step 4: Device Posture Integration

Integrate endpoint security tools to monitor device health and compliance.

Step 5: Policy Configuration

Create context-aware policies that define access conditions based on identity, device, location, and application risk.

Step 6: Client Connector Deployment

Install the Zscaler Client Connector on user devices to enable secure application access.

Benefits of Context-Aware Access Control

Create context-aware policies that define access conditions based on identity, device, location, and application risk.

Step 6: Client Connector Deployment

Install the Zscaler Client Connector on user devices to enable secure application access.

Benefits of Context-Aware Access Control

Implementing context-aware access control provides several security advantages.

- Reduces the attack surface by restricting access to specific applications
- Prevents lateral movement inside the network
- Strengthens authentication using multiple security signals
- Protects sensitive data through risk-based access policies
- Provides better visibility into user activity and access patterns

This approach significantly improves the organization's overall cybersecurity posture.

Conclusion Stage

1

focused on identifying web application vulnerabilities using OWASP security standards. Common vulnerabilities such as SQL injection, path traversal, and cryptographic failures were analysed to understand their impact on application security.

Stage 2

Involved performing vulnerability scanning using the Nessus tool. The scan results highlighted security issues such as outdated encryption protocols and misconfigured system settings. Recommendations were provided to improve the security configuration of the system.

Stage 3

Concentrated on designing and applying context-aware access controls using Zero Trust principles. By leveraging Zscaler Private Access, the system verifies user identity, device posture, and location before granting access to applications.

Concentrated on designing and applying context-aware access controls using Zero Trust principles. By leveraging Zscaler Private Access, the

system verifies user identity, device posture, location, and application risk before granting access. This modern security model replaces traditional network-based trust with identity-centric security, significantly reducing the risk of unauthorized access and cyber-attacks.

Future Scope

The implementation of context-aware access control opens several opportunities for further improvements in cybersecurity.

Future enhancements may include:

- Integration with Secure Access Service Edge (SASE) platforms
- AI-based anomaly detection for identifying suspicious behaviour
- Advanced device risk scoring using endpoint security tools
- Automated access provisioning and revocation based on user roles
- Extending Zero Trust security to cloud workloads and IoT devices

These advancements will further strengthen security and enable organizations to build intelligent and adaptive cybersecurity systems.

Topics Explored:

- Zero Trust Security Model
- Context-Aware Access Control
- Identity-Based Security
- Device Posture Verification
- Risk-Based Application Access
- Threat Intelligence
- Incident Response

Tools Explored:

- Zscaler Private Access (ZPA)
- Nessus Vulnerability Scanner
- OWASP Security Standards